

Simple linear regression

```
In [1]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
%matplotlib inline
```

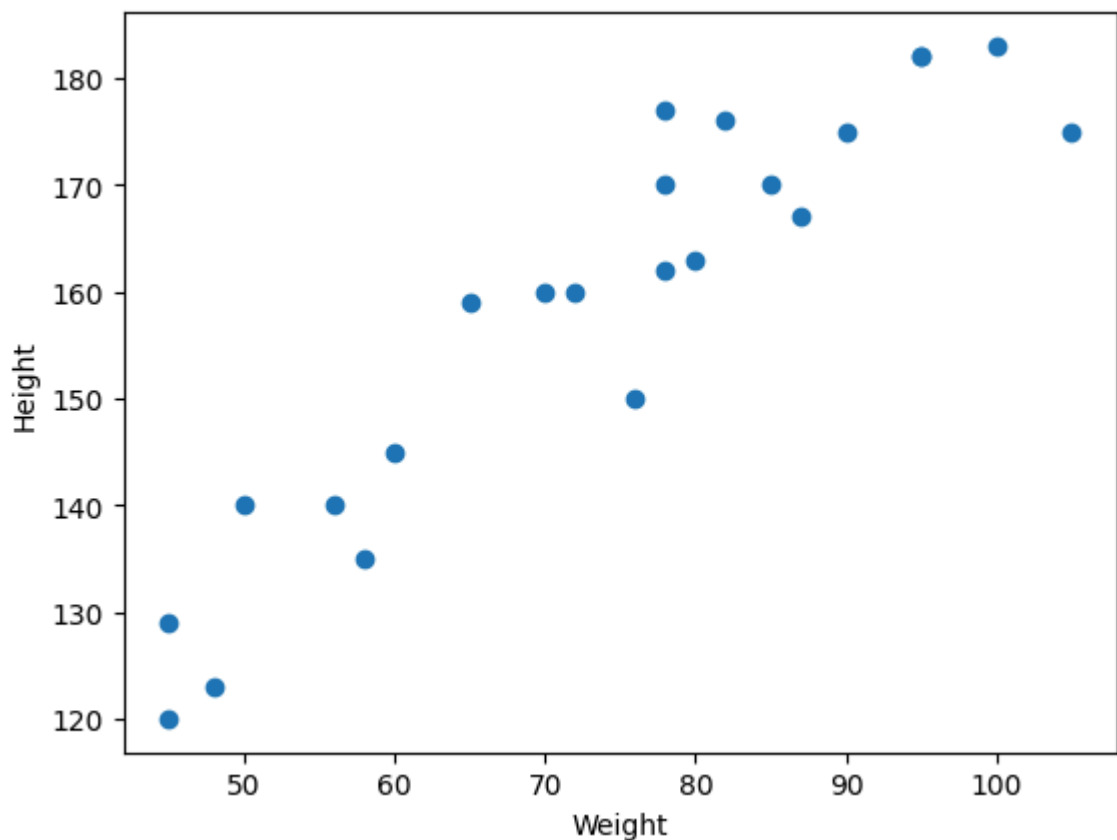
```
In [2]: df=pd.read_csv('/Users/mujeebmohammed/Downloads/height-weight.csv')
df.head()
```

Out[2]:

	Weight	Height
0	45	120
1	58	135
2	48	123
3	60	145
4	70	160

```
In [3]: ##scatter plot
plt.scatter(df['Weight'],df['Height'])
plt.xlabel('Weight')
plt.ylabel('Height')
```

Out[3]: Text(0, 0.5, 'Height')



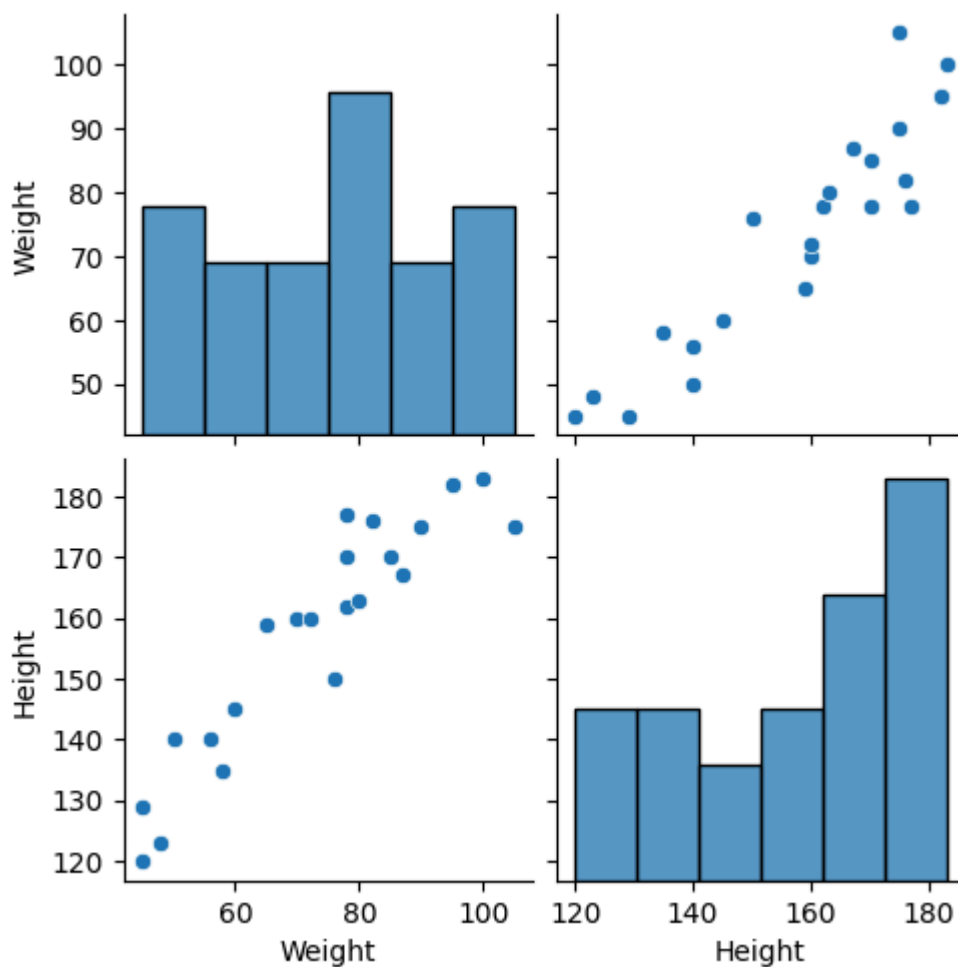
```
In [4]: #correation
df.corr()
```

```
Out[4]:
```

	Weight	Height
Weight	1.000000	0.931142
Height	0.931142	1.000000

```
In [5]: #seaborn for visualization
import seaborn as sns
sns.pairplot(df)
```

```
Out[5]: <seaborn.axisgrid.PairGrid at 0x120396510>
```



```
In [6]: ## independent and dependent features
X=df[['Weight']]    ###independent features should be data frame or 2d a
y=df['Height']      ###this variable can be in series or 1d array
```

```
In [7]: X_series=df['Weight']
np.array(X_series).shape
```

```
Out[7]: (23,)
```

```
In [8]: np.array(y).shape
```

```
Out[8]: (23,)
```

```
In [9]: ## train test split  
from sklearn.model_selection import train_test_split
```

```
In [10]: X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.25,random_
```

```
In [11]: X_train.shape
```

```
Out[11]: (17, 1)
```

```
In [12]: ##standardization  
from sklearn.preprocessing import StandardScaler
```

```
In [13]: scaler=StandardScaler()  
X_train=scaler.fit_transform(X_train)
```

```
In [14]: X_test=scaler.transform(X_test)
```

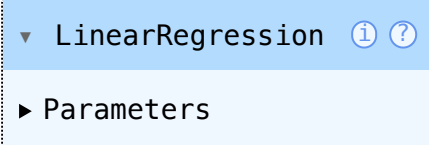
```
In [15]: X_test
```

```
Out[15]: array([[ 0.33497168],  
                [ 0.33497168],  
                [-1.6641678 ],  
                [ 1.36483141],  
                [-0.45256812],  
                [ 1.97063125]])
```

```
In [16]: ### apply simple linear regression  
from sklearn.linear_model import LinearRegression
```

```
In [17]: regression=LinearRegression()
```

```
In [18]: regression.fit(X_train,y_train)
```

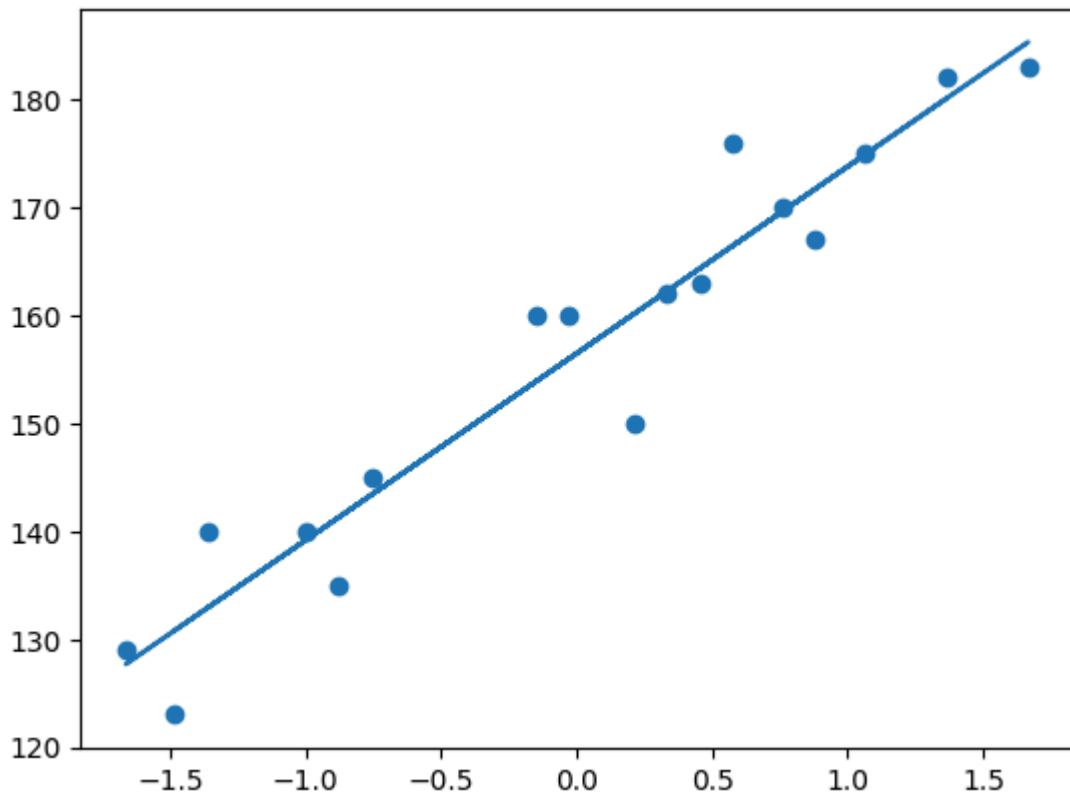
```
Out[18]:  LinearRegression ⓘ ?  
▶ Parameters
```

```
In [19]: print('coefficients or slope:',regression.coef_)  
print('intercept:',regression.intercept_)
```

```
coefficients or slope: [17.2982057]  
intercept: 156.47058823529412
```

```
In [20]: # plot training data point best fit line  
plt.scatter(X_train,y_train)  
plt.plot(X_train,regression.predict(X_train))
```

```
Out[20]: [ at 0x1285e6350>]
```



```
In [21]: #prediction of test data
#1. predicted height output=intercept +coef_(weights)
#2. y_pred_test=156.470+17.29(X_test)
```

```
In [22]: #prediction for test data

y_pred=regression.predict(X_test)
y_pred
```

```
Out[22]: array([162.26499721, 162.26499721, 127.68347133, 180.07972266,
               148.64197186, 190.55897293])
```

```
In [23]: ## performance metrics
from sklearn.metrics import mean_squared_error,mean_absolute_error
```

```
In [24]: mse=mean_squared_error(y_test,y_pred)
mae=mean_absolute_error(y_test,y_pred)
rmse=np.sqrt(mse)
print(mse)
print(mae)
print(rmse)
```

```
114.84069295228699
9.66512588679501
10.716374991212605
```

```
In [25]: # r2

from sklearn.metrics import r2_score
r2=r2_score(y_test,y_pred)
print(r2)
```

```
0.7360826717981276
```

```
In [26]: # display adjusted r2
1-(1-r2)*(len(y_test)-1)/(len(y_test)-X.shape[1]-1)
```

```
Out[26]: 0.6701033397476595
```

```
In [27]: pip install statsmodels
```

Requirement already satisfied: statsmodels in /Users/mujeebmohammed/Documents/DS/Python/venv/lib/python3.11/site-packages (0.14.6)
Requirement already satisfied: numpy<3,>=1.22.3 in /Users/mujeebmohammed/Documents/DS/Python/venv/lib/python3.11/site-packages (from statsmodels) (2.4.2)
Requirement already satisfied: scipy!=1.9.2,>=1.8 in /Users/mujeebmohammed/Documents/DS/Python/venv/lib/python3.11/site-packages (from statsmodels) (1.17.0)
Requirement already satisfied: pandas!=2.1.0,>=1.4 in /Users/mujeebmohammed/Documents/DS/Python/venv/lib/python3.11/site-packages (from statsmodels) (3.0.0)
Requirement already satisfied: patsy>=0.5.6 in /Users/mujeebmohammed/Documents/DS/Python/venv/lib/python3.11/site-packages (from statsmodels) (1.0.2)
Requirement already satisfied: packaging>=21.3 in /Users/mujeebmohammed/Documents/DS/Python/venv/lib/python3.11/site-packages (from statsmodels) (26.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /Users/mujeebmohammed/Documents/DS/Python/venv/lib/python3.11/site-packages (from pandas!=2.1.0,>=1.4->statsmodels) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /Users/mujeebmohammed/Documents/DS/Python/venv/lib/python3.11/site-packages (from python-dateutil>=2.8.2->pandas!=2.1.0,>=1.4->statsmodels) (1.17.0)
Note: you may need to restart the kernel to use updated packages.

```
In [28]: ## ols linear regression
import statsmodels.api as sm
```

```
In [29]: model=sm.OLS(y_train,X_train).fit()
```

```
In [30]: prediction=model.predict(X_test)
print(prediction)
```

```
[ 5.79440897  5.79440897 -28.78711691  23.60913442 -7.82861638
 34.08838469]
```

```
In [31]: print(model.summary())
```

```

                                OLS Regression Results
=====
Dep. Variable:                  Height    R-squared (uncentered):
0.012
Model:                          OLS      Adj. R-squared (uncentered):
-0.050
Method:                        Least Squares    F-statistic:
0.1953
Date:                          Sat, 14 Mar 2026    Prob (F-statistic):
0.664
Time:                          00:28:06    Log-Likelihood:
-110.03
No. Observations:              17    AIC:
222.1
Df Residuals:                  16    BIC:
222.9
Df Model:                      1
Covariance Type:              nonrobust
=====
=====

```

	coef	std err	t	P> t	[0.025	0.
x1	17.2982	39.138	0.442	0.664	-65.671	10

```

-----
=====
Omnibus:                      0.135    Durbin-Watson:
0.002
Prob(Omnibus):                0.935    Jarque-Bera (JB):
0.203
Skew:                         -0.166    Prob(JB):
0.904
Kurtosis:                     2.581    Cond. No.
1.00
=====
=====

```

Notes:

- [1] R^2 is computed without centering (uncentered) since the model does not contain a constant.
- [2] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In [32]: `## prediction for new data`

```
regression.predict(scaler.transform([[72]]))
```

```

/Users/mujeebmohammed/Documents/DS/Python/venv/lib/python3.11/site-packages/sklearn/utils/validation.py:2691: UserWarning: X does not have valid feature names, but StandardScaler was fitted with feature names
  warnings.warn(

```

Out[32]: `array([155.97744705])`

In []: